
Panel

An interview engine where every score has to cite the transcript, and the criteria are frozen before a single question is asked.

TECHNICAL DOCUMENTATION

VERSION 0.1.0 · 19 AUGUST 2026

5,901 LINES · 47 SOURCE FILES · 81 TESTS

PYTHON 3.12 · REACT 18 · SQLITE

CONTENTS

What is in this document

01	What Panel is The two modes, and the one engine underneath
02	Why it works this way The research that set the design
03	The three rules Frozen criteria, cited scores, honest gaps
04	Architecture Transport-agnostic core, replaceable edges
05	The numbers Size, tests, coverage, dependencies
06	Every file explained All 47 source files, and why each exists
07	The rubric library 9 competencies, 36 behavioural anchors
08	The data model Where the rules are enforced by types
09	The HTTP API 9 routes, and what each guarantees
10	The web interface Design decisions and what they encode
11	Testing What 81 tests actually assert
12	Bugs found while building Four real defects, and how each surfaced
13	Status Verified, scaffolded, and not built
14	Running it Install, commands, ports, keys
15	Appendix Commit history and glossary

01

What Panel is

Panel conducts a structured interview, then produces a score in which every judgement points at the words that produced it.

It runs in two modes over one engine. In **practice mode** you are the candidate: you answer, and the output is coaching that tells you what a stronger answer would have contained. In **screening mode** somebody else is the candidate and the output is a scorecard, with each score attached to the quotes that justify it.

The two modes are not two products. They share the plan compiler, the conductor, the evidence extractor and the scorer; they differ only in the final rendering. That is deliberate — the moment coaching and screening have separate scoring paths, they start disagreeing about what a level 3 answer looks like.

The thing that makes it different

There is no shortage of AI interview tools. Nearly all of them share a weakness: a model reads an answer and emits a number, and nobody can say afterwards what the number was based on. If you disagree with a 2 out of 4, there is nothing to inspect.

Panel is built so that question cannot be dodged. Criteria are compiled and content-hashed *before* the interview starts, so the interviewer cannot drift onto easier ground mid-conversation. A score without a supporting quote is rejected by the type system, not discouraged by a prompt. And a competency the interview never reached is recorded as *not observed*, kept out of the average entirely, rather than quietly counted as a zero.

THE SHAPE OF IT

Compile a frozen plan from the role, résumé and job description. Conduct the interview inside that plan, probing where answers are thin. Extract evidence as you go, tying every claim to a transcript span. Score against behavioural anchors afterwards, citing that evidence. Render it two ways.

Who it is for

This is a personal tool, built for one person to practise interviews and occasionally screen someone. It is not a hiring platform. That scope is load-bearing: it is why sessions live in memory, why there is no multi-tenancy, and why the compliance posture is *build the audit surface in now because retrofitting it is painful* rather than *ship a compliance product*.

Why it works this way

One published finding shaped almost every structural decision in this codebase.

Before writing any code, the question worth answering was: what actually makes an LLM's judgement of an interview answer reliable? The literature on LLM-as-judge scoring is unusually clear about this, and the answer is not the model.

SCORING SETUP	AGREEMENT WITH HUMAN EXPERTS
Unanchored — “rate this 1 to 5”	$r \approx 0.20$ — effectively noise
Behaviourally-anchored rubric	$r \approx 0.63$
Anchors plus annotated critical points	> 0.89 agreement, even from cheaper judge models

The jump from 0.20 to 0.63 comes from replacing adjectives with observable behaviour. “Shows good judgement” cannot be checked against a transcript; “names the tradeoff they accepted and what it cost” can. The jump to 0.89 comes from telling the judge, in advance, what a strong answer contains.

So quality lives in the rubric layer, not the model. Everything else follows from taking that seriously.

What that bought

Because the rubric is where the quality is, it made sense to make the rubric a first-class, versioned object rather than prompt text. That one decision produced the content hash, which produced comparability between runs, which is what later made progress tracking meaningful. None of that was planned up front; it fell out of putting the rubric at the centre.

The market

The candidate-practice space is crowded — Final Round AI, OphyAI, Pramp/Exponent, Interviewing.io — but almost entirely closed SaaS. There is no strong open reference implementation, which made building one worthwhile rather than redundant.

The legal picture

Personal use sits outside both regimes, but they shaped the design anyway. The EU AI Act classes recruitment and screening AI as **high-risk**, with obligations applying from 2 December 2027. New York City Local Law 144 already requires an independent annual bias audit, a public summary, and ten business days' notice to candidates, with enforcement tightened during 2026.

ONE FEATURE DELIBERATELY EXCLUDED

No webcam affect, emotion, or body-language scoring. It is the weakest-evidence part of commercial AI interviewing, and the EU AI Act bans emotion recognition in the workplace. Video in Panel exists for realism and self-review; it is never an input to a score. The camera stream never leaves the browser.

03

The three rules

Each is enforced by structure — a type, a schema, a hash — rather than requested politely in a prompt.

Rule one — criteria are frozen before the interview

The plan compiler turns a role, an interview type, and optionally a résumé and job description into an **InterviewPlan**: competencies, their rubric anchors, the critical points a strong answer contains, and the planned questions. That object is then content-hashed.

```
plan_hash = sha256(role + type + competencies + questions)[:16]
            → '70bc65ce6c91efcb'
```

During the interview the conductor may choose *which* planned question to ask and *how deep* to probe. It cannot add a competency, invent a question, or edit an anchor. Adaptivity happens inside a fixed frame. Two interviews carrying the same hash were assessed against byte-identical criteria — which is what makes their scores comparable at all.

Rule two — no citation, no score

A **CompetencyScore** with a numeric level and no supporting evidence raises a validation error. It is not possible to construct one. When the LLM path returns evidence, each quote is checked against the transcript and dropped if it is not found verbatim.

WHY VERIFY THE QUOTES

A fabricated citation is worse than no citation. A missing quote is visibly missing; an invented one makes an unsupported score look auditable. The check is cheap, so it runs on every piece of evidence.

Rule three — not observed is not zero

If the interview never gathered evidence for a competency, its level is **None**. It is excluded from the overall average rather than counted as zero, reported separately in both renderings, and coverage is always shown alongside the score.

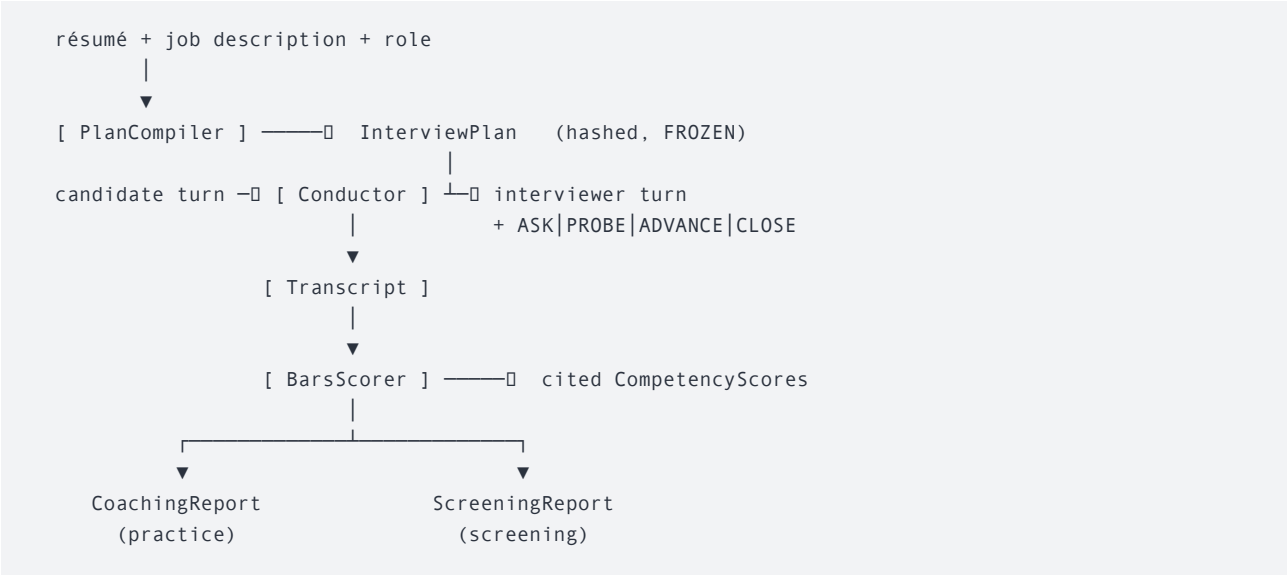
The distinction matters because the two things have opposite meanings. A low score says the candidate answered badly. Not observed says *the interview failed to ask*. Averaging them together blames the candidate for the interviewer's omission.

This rule holds all the way down. In the SQLite schema the level column is nullable, so **AVG()** skips it. The domain model refuses to average an unobserved competency as zero; the database refuses too. The rule is not re-argued at each layer.

Architecture

The valuable part is transport-agnostic. Audio, video and avatars wrap it; they never leak inward.

The engine consumes a string of candidate speech and returns a string for the interviewer to say, plus the decision behind it. It knows nothing about sockets, audio or browsers. Everything else is an adapter.



Why this shape

- The whole engine is testable with no audio, no network and no API key — the suite runs in under two seconds.
- Rubric iteration, which sets the quality ceiling, costs nothing.
- Cheap by default: the text path bills nothing, and the avatar is an explicit opt-in rather than a default.
- A live-coding interview later becomes a new plan type, not a rewrite.

Three transports, one engine

TRANSPORT	STATUS	WHAT IT DOES
Text	Verified	Terminal interview. Nine lines of loop, because the engine does the work.
HTTP	Verified	FastAPI session API behind the browser UI. Holds sessions, moves text.
Realtime	Scaffolded	LiveKit voice with optional avatar. Never run against a live room.

The four decisions

After every candidate answer the conductor returns exactly one of these. The set is deliberately small — a state machine with four moves is one you can reason about, and every move stays inside the frozen plan.

DECISION	WHEN	WHAT IT MEANS
ASK	The answer covered its critical points, or probing is spent	Move to the next planned question in the same competency
PROBE	Points are missing and the probe cap is not reached	Follow up on this answer, targeting the most valuable gap
ADVANCE	This competency's questions or time share are done	Cross into the next competency
CLOSE	The plan is exhausted or the global budget is spent	End the interview and score it

THE REALTIME DESIGN DECISION

LiveKit's standard pattern is speech-to-text → LLM → text-to-speech, with the model deciding what to say. That is exactly wrong here: the conductor has already decided, from a frozen plan, and a model in the loop would improvise questions outside it. AgentSession takes the LLM as optional, so the wiring is STT → Conductor → TTS. The reasoner still runs — for judging and scoring — just off the speech path where its latency cannot stall the conversation.

05

The numbers

Everything below was measured, not estimated.

5,901
TOTAL LINES

47
SOURCE FILES

81
TESTS

1.3s
SUITE RUNTIME

4
COMMITTS

Where the code lives

AREA	FILES	LINES	WHAT IT IS
Python engine	24	3,297	Models, rubric library, compiler, conductor, scoring, storage, transports, CLI, API
Tests	6	1,148	81 tests across models, engine, scoring, API, storage, realtime gate
Web app	9	1,456	React video-call UI, history view, stylesheet
Total	39	5,901	Plus configuration: pyproject, package.json, launch.json, dev.sh, README, design spec

The rubric library in numbers

9
COMPETENCIES

36
BEHAVIOURAL ANCHORS

36
CRITICAL POINTS

18
BANK QUESTIONS

Five behavioural competencies — ownership, handling disagreement, structured problem solving, learning from failure, communication. Four technical — systems design tradeoffs, debugging methodology, depth of fundamentals, code quality judgement. Each carries four anchors and four critical points.

Dependencies

GROUP	PACKAGES	NEEDED FOR
runtime	pydantic, anthropic, rich	The engine and terminal interface
api	fastapi, uvicorn	The HTTP transport and web UI
dev	pytest, httpx	The test suite
realtime	livekit-agents and three plugins	Voice — optional, never required

The runtime set is deliberately small. Nothing in the realtime or API groups is needed to run an interview in the terminal or to run the tests.

Every source file by size

The full Python engine, largest first. The rubric library leads because that is where the quality ceiling is set, not because it is complicated.

FILE	LINES	ROLE
panel/planning/library.py	368	The rubric library — 9 competencies, 36 anchors
panel/api/app.py	327	HTTP transport, 9 routes, session store
panel/llm/heuristic.py	315	Keyless reasoner — signal detection
panel/models.py	292	Domain model; two rules enforced as types
panel/llm/anthropic.py	289	Claude-backed reasoner with quote verification
panel/engine/conductor.py	265	The interview state machine
panel/cli.py	233	Terminal interface, five commands
panel/scoring/report.py	230	One report object, several renderings
panel/storage/db.py	220	Append-only SQLite store
panel/transport/realtime.py	217	LiveKit voice transport (unverified)
panel/demo_answers.py	128	Scripted answers, keyed by competency
panel/planning/compiler.py	100	Builds and freezes the plan
panel/llm/base.py	76	The Reasoner protocol
panel/scoring/scorer.py	74	Scores against anchors, cites evidence
panel/transport/text.py	44	Terminal transport — nine lines of loop
panel/config.py	34	Settings; demo mode when no key is present
seven __init__.py files	62	Public surfaces for each package

06

Every file explained

All 47 source files, grouped by package, with what each does and why it exists rather than being folded into something else.

panel/ — the root

Configuration and the domain model everything else depends on.

FILE	LINES	WHAT IT DOES AND WHY
models.py	292	The domain model, and where two of the three rules become unbreakable. CompetencyScore raises if given a level with no supporting evidence. Competency raises unless it carries exactly four anchors. InterviewPlan.plan_hash content-hashes the criteria, excluding id and timestamp so identical criteria hash identically. Also holds Transcript, Turn, Evidence and the ScoredInterview aggregate whose overall() excludes unobserved competencies rather than zeroing them.
config.py	34	Settings from the environment. The important line is demo_mode : no API key means the heuristic reasoner runs and everything still works. max_tokens defaults high because thinking is on by default on Opus 5 and the budget covers thinking plus output.
cli.py	233	Five commands — practice, screen, demo, history — over the same engine. Renders reports with rich, records finished interviews, and tells you when a run is comparable to earlier ones.
demo_answers.py	128	Seventeen scripted answers across nine competencies, keyed by competency rather than held in a list. A positional script drifts out of alignment the moment a probe fires. Deliberately uneven — strong answers, thin ones, and one genuine non-answer — so a demo exercises every branch of the conductor.

panel/planning/ — compiling the plan

Turns a role and some documents into frozen criteria.

FILE	LINES	WHAT IT DOES AND WHY
library.py	368	The vetted rubric library, and the highest-value file in the repository. Nine competencies, each with four behavioural anchors and four critical points. Every descriptor describes something observable in an answer — never a trait. This is where the $r=0.63$ lives.
compiler.py	100	Builds the InterviewPlan and freezes it. With a reasoner and a résumé or job description, questions are drafted against the real role; without, the vetted bank is used. The rubric is identical either way — only questions become role-specific, because the rubric is what must stay stable for scores to mean anything.
__init__.py	4	Public surface: compile_plan, competencies_for.

panel/engine/ — conducting the interview

The state machine. Transport-agnostic by construction.

FILE	LINES	WHAT IT DOES AND WHY
conductor.py	265	Consumes candidate text, returns interviewer text plus a decision — ASK, PROBE, ADVANCE or CLOSE. Tracks evidence coverage per competency. Carries two budgets: a global exchange ceiling and a per-competency share, the second added after the first build proved depth-first and never reached the later competencies. Exposes Progress and current_question for any UI to render.
__init__.py	3	Exports Conductor and Step.

panel/llm/ — judgement calls

A domain-level interface, not a raw prompt wrapper. That is what lets the keyless implementation be a real peer rather than a stub.

FILE	LINES	WHAT IT DOES AND WHY
base.py	76	The Reasoner protocol: draft_questions, assess_answer, score, coach. Deliberately domain-level — the conductor asks “was this answer substantive and what did it miss?”, never “here is a prompt”. Keeps prompt engineering out of the state machine.
heuristic.py	315	The keyless reasoner. Not an imitation of a language model: it detects the six observable signals the rubrics are written around — specificity, first-person agency, stated outcomes, considered alternatives, reflection, method — and reports honestly on what it cannot see. Spelled-out numerals count as specificity because speech-to-text writes “two days”, not “2 days”.
anthropic.py	289	The Claude-backed reasoner. Structured output through messages.parse() with Pydantic models, so the engine never parses free text. Verifies every quote against the transcript and drops unverifiable ones. Any API failure degrades to the heuristic rather than crashing an interview in progress.
__init__.py	25	get_reasoner() — picks the Claude reasoner when a key exists, the heuristic otherwise.

panel/scoring/ — judging and rendering

Separate from conducting on purpose: during the interview the engine optimises for a good conversation, here for a defensible judgement.

FILE	LINES	WHAT IT DOES AND WHY
scorer.py	74	Scores each competency against its anchors using only extracted evidence. A competency with no supporting evidence returns NOT OBSERVED without consulting the reasoner at all — and if a reasoner returns a level anyway, it is discarded.
report.py	230	One Report object, several renderings. build_report is the source of truth; the CLI renders text from it, the API serialises it as JSON. Refactored to this shape when the browser arrived, so the two surfaces cannot drift about what a score means.
__init__.py	21	Public surface for scoring and reports.

panel/storage/ — the record

Append-only, because a scorecard you can quietly edit afterwards is not a record.

FILE	LINES	WHAT IT DOES AND WHY
db.py	220	SQLite store. Two tables: interviews (summary plus the full report JSON) and competency_scores (one row per competency, level nullable). No update or delete path exists. trends() is scoped to a single plan_hash, because interviews compiled from different criteria are not comparable and charting them together would hide that.
__init__.py	23	Public surface for the store.

panel/transport/ — the edges

Adapters. They move text in and out; they make no decisions.

FILE	LINES	WHAT IT DOES AND WHY
text.py	44	The terminal transport. Nine lines of actual loop, because the engine does the work. A realtime transport substitutes different say and listen callables and changes nothing else.
realtime.py	217	LiveKit voice with optional avatar. Written against livekit-agents 1.6.10 with the API verified by introspection, but never run against a live room. Contains the credential gate, which is tested, and the call path, which is not.
__init__.py	3	Exports run_interview.

panel/api/ — the HTTP transport

A transport like the CLI. Every interview decision stays in the engine.

FILE	LINES	WHAT IT DOES AND WHY
app.py	327	FastAPI application: nine routes, in-memory session store, SQLite connection opened on the app lifespan. Persists an interview when it finishes, whether or not anyone asks to see the report.
__init__.py	3	Exports the app.

web/src/ — the browser interface

React 18 on Vite 5. Hand-written CSS with custom properties; no framework.

FILE	LINES	WHAT IT DOES AND WHY
App.jsx	170	Phase machine — setup, running, done, history — plus the timer, the demo-mode banner and the topbar carrying the rubric hash.
components/Setup.jsx	132	Role, focus, length, mode, and optional résumé and job description.
components/Call.jsx	120	The video-call stage: interviewer tile, your camera, the current question in serif, the composer, and a side rail with progress and live transcript.
components/Selfie.jsx	44	Your camera via getUserMedia, attached to a local video element. Never uploaded, never scored. Degrades to “Camera off” when denied.

FILE	LINES	WHAT IT DOES AND WHY
components/Report.jsx	176	The report on paper. Holds the signature interaction: clicking a citation lights and scrolls to the transcript turn it came from.
components/History.jsx	147	Past interviews and per-competency progress, scoped to one rubric version with that constraint stated on the page.
api.js	45	Thin fetch client. Surfaces the server's own message, so a 409 reads as “this interview is already finished” rather than “request failed”.
styles.css	612	The whole design system: ink call, paper report, one amber accent, three type roles, responsive down to 375px.
main.jsx	10	React root.

tests/ — 81 tests

The suite runs in 1.3 seconds with no key and no network.

FILE	LINES	WHAT IT DOES AND WHY
test_models.py	174 · 14 tests	The rules as types: rubrics must have four anchors, scores must cite, hashes must be stable, unobserved must not be averaged as zero.
test_engine.py	217 · 12 tests	Conductor transitions with a stub reasoner, so decision logic is the only variable. Includes the breadth-before-depth budget guarantee.
test_scoring.py	231 · 13 tests	Heuristic signal detection, scorer rules, and a full end-to-end interview asserting every scored competency cites a real quote.
test_api.py	254 · 24 tests	The HTTP contract, including that history records a finished interview without being asked and that an abandoned one is not recorded.
test_storage.py	196 · 12 tests	Append-only guarantees, NULL semantics surviving into SQL, and trends excluding other rubric versions.
test_realtime_gate.py	76 · 6 tests	The credential gate — the only part of the realtime transport testable without a live room.

Configuration and documentation

FILE	LINES	WHAT IT DOES AND WHY
pyproject.toml	—	Package metadata and four dependency groups: runtime, api, dev, realtime.
dev.sh	—	Runs the API on 8040 and the web app on 5193 together, with the API killed on exit.
.claude/launch.json	—	Preview configuration for the dev server.
web/vite.config.js	—	Vite 5 on port 5193, proxying /api to 8040.
README.md	—	Orientation, the three rules, usage, and an explicit statement of what is verified versus scaffolded.
docs/.../panel-design.md	—	The design document, including decisions made mid-build and why.

07

The rubric library

Nine competencies, thirty-six anchors. Every descriptor describes something observable in an answer — never a trait of the person.

This is the file that sets the quality ceiling, so it is worth seeing what an anchor actually looks like. Below is the complete rubric for **Ownership**, one of five behavioural competencies.

LEVEL	OBSERVABLE IN THE ANSWER
1 — Weak	Speaks only in the plural about team activity; no personal action is identifiable. Or names a responsibility held without any instance of exercising it.
2 — Developing	Names a personal action, but it stays inside assigned duties. Ownership stops at the task boundary; hand-offs are described as the end of their involvement.
3 — Strong	Names a specific problem they took on that nobody assigned them, the action they personally took, and the outcome it produced.
4 — Exceptional	As Strong, and they changed the system rather than the instance — a process, guardrail, test, or norm that outlived the incident and prevented recurrence.

Each competency also carries **critical points** — the concrete things a strong answer contains. These drive both probing during the interview and scoring afterwards. For Ownership: a specific incident rather than a general practice; their own action stated separately from the team's; an outcome that is observable or measurable; evidence they went past what was assigned.

The full set

COMPETENCY	TYPE	WHAT IT ASSESSES
Ownership	behavioural	Responsibility beyond assigned tasks
Handling Disagreement	behavioural	Navigating conflict toward a decision
Structured Problem Solving	behavioural	Decomposing ambiguity methodically
Learning From Failure	behavioural	Extracting transferable lessons
Communication	behavioural	Conveying complex work to a listener
Systems Design Tradeoffs	technical	Reasoning about what each option costs
Debugging Methodology	technical	Isolating cause systematically, not by guesswork
Depth of Fundamentals	technical	Understanding what sits under the abstractions
Code Quality Judgement	technical	When structure is worth it and when it is waste

A behavioural interview draws the five behavioural competencies; a technical-verbal interview the four technical ones; a mixed interview takes the behavioural core plus the two highest-signal technical ones.

08

The data model

Where the rules stop being conventions and become things the program cannot express.

TYPE	WHAT IT GUARANTEES
RubricLevel	One behaviourally-anchored level: a number 1–4, a label, and a descriptor of at least ten characters describing observable behaviour.
Competency	Raises unless it carries exactly levels 1 through 4. A three-anchor or duplicate-level rubric cannot be constructed.
InterviewPlan	Validates that every question references a known competency and that at least one question exists. Exposes plan_hash — sha256 over the criteria, excluding id and timestamp.
Transcript / Turn	Append-only conversation with stable integer indices. Turn indices are what citations point at.
Evidence	A claim tied to (competency, turn index, quoted span), with polarity either supporting or undermining.
CompetencyScore	Raises if given a level with no supporting evidence. level=None means not observed, which is a legitimate and distinct outcome.
ScoredInterview	overall() is a weighted mean over observed competencies only; coverage is reported separately so a thin interview is visible rather than hidden behind a confident number.

THE POINT OF PUTTING IT IN TYPES

A prompt asking a model to “always cite evidence” is a request. A constructor that refuses to build the object is a guarantee. Every layer downstream — the scorer, the reports, the API, the database — inherits that guarantee without having to re-check it.

09

The HTTP API

Nine routes. FastAPI, with Pydantic models on both the request and response side.

METHOD	ROUTE	WHAT IT DOES
GET	/api/health	Which reasoner is live, and an explicit note that sessions are held in memory
POST	/api/sessions	Compiles a frozen plan, opens a conductor, returns the plan summary and first question
POST	/api/sessions/{id}/answer	One candidate turn in, one interviewer move out, with progress. 409 if already finished
GET	/api/sessions/{id}	Current state and transcript — survives a page reload
GET	/api/sessions/{id}/report	The scored report. 409 while the interview is still running
DELETE	/api/sessions/{id}	Drops the in-memory session; a finished interview stays in the store
GET	/api/history	Past interviews, newest first, optionally filtered by role
GET	/api/history/{id}	A past report, citations intact
GET	/api/trends/{plan_hash}	Per-competency progress across runs of one rubric version only

Two behaviours worth knowing

- **Finishing records the interview.** Persistence happens when the conductor closes, not when someone requests the report — so an interview you never look at is still in your history, and an abandoned one is not.
- **Trends refuse to mix rubrics.** The route is keyed by plan_hash and returns a note saying so. Comparability is what freezing the plan bought; quietly averaging across criteria would spend it for nothing.

The web interface

The structural decision: the call is dark, the report is paper. Two phases, two genuinely different objects.

A live conversation and a filed record are not the same kind of thing, and the interface says so by inverting between them. During the call you are in a dark video-call surface. When it ends you are looking at a document.

The design system

ELEMENT	CHOICE	WHY
Call ground	ink #11151C	Video-call surface; attention on the exchange
Report ground	paper #E9EBEE	A cool printed-script white, not the warm cream default
Accent	amber #C98A2B	Used only for citation marks and the live indicator
Not observed	mute #7A8494	Hollow dashed scale — never reads as a low score
Spoken question	Instrument Serif	The words being said aloud, set apart from the interface
Interface	IBM Plex Sans	Technical without being Inter
The record layer	IBM Plex Mono	Turn indices, rubric hashes, timers — true to the content

The signature interaction

Every piece of evidence in a report is a button carrying its turn index. Clicking it lights the source turn in the full transcript below and scrolls it into view. A score can always be walked back to the words that produced it, in one click.

This was verified on a report that had been serialised to SQLite and restored: citation T25 still landed on turn T25, with the quoted text present in that turn. The audit trail survives persistence, which is the whole claim.

WHAT THE CAMERA IS FOR

Your camera appears in the corner of the call for realism and self-review. The stream is attached to a local video element and never leaves the browser — not uploaded, not recorded, not scored. When permission is denied the tile reads “Camera off” and the interview continues unaffected.

11

Testing

81 tests in 1.3 seconds, with no API key and no network. That speed is the payoff from keeping the engine transport-agnostic.

81	6	1,148	1.3s
TESTS	FILES	LINES	RUNTIME

The tests are written against behaviour rather than implementation, and several of them exist specifically to stop a rule from quietly eroding.

Tests that guard the three rules

TEST	WHAT WOULD BREAK WITHOUT IT
test_score_without_evidence_is_rejected	A score could be emitted with nothing behind it
test_undermining_evidence_alone_does_not_justify_a_score	Counter-evidence could be miscounted as support
test_unobserved_excluded_from_overall_not_counted_as_zero	A thin interview would read as a bad candidate
test_hash_changes_when_an_anchor_changes	Criteria could drift while still claiming comparability
test_a_fabricated_level_is_downgraded_not_trusted	A misbehaving reasoner could inject an uncited score
test_probes_never_echo_internal_rubric_wording	The interviewer would read the answer key aloud
test_every_competency_gets_asked...	The conductor would exhaust its budget on the first topic
test_sql_average_skips_unobserved_rather_than_counting_zero	The NULL rule would hold in Python and break in SQL

A test design decision worth noting

The end-to-end test originally asserted 100% coverage, which conflated two different responsibilities. It now hard-asserts the **engine** invariant — the conductor must ask about every competency it planned to assess — and holds **reasoner recall** to a separate, lower bar. The conductor's job is to ask; extraction quality is somebody else's job, and the keyless reasoner is explicitly held to a weaker standard than the LLM path.

12

Bugs found while building

Four real defects. Three were invisible to the test suite and only appeared when the thing was actually run.

01 The interviewer read the answer key aloud

Symptom	A probe rendered as: “Say more about this: evidence they went past what was assigned.” That is an internal rubric string — it tells the candidate exactly which box to tick.
Fix	Probes now come from a bank of interviewer-voiced follow-ups, rotated so a long interview never repeats one verbatim. A regression test asserts no probe ever contains a critical point’s own wording.
Found by	<i>Running the demo and reading the output</i>

02 The conductor was depth-first

Symptom	It probed the opening competency until the clock ran out and never asked about the last three. A 30-minute mixed interview assessed 2 of 5 competencies — producing a confident score on one area and NOT OBSERVED everywhere else.
Fix	A per-competency budget alongside the global one. Breadth first, depth second: an interview that covered two of five competencies is a worse interview however good those two scores are.
Found by	<i>Instrumenting a run and printing what was actually asked</i>

03 Nothing scrolled inside the page

Symptom	The app shell used min-height, so it grew to 7,503 pixels. The report’s overflow never engaged, the whole document scrolled as one column, and the topbar carrying the rubric hash scrolled away.
Fix	The shell is now exactly viewport height, so the call and the report each scroll inside their own box.
Found by	<i>Measuring element geometry in the browser</i>

04 The citation jump was silently dead

Symptom	Clicking a citation highlighted the right turn but never moved the page — losing half the feature. The first fix made it worse: adding CSS scroll-behavior broke it again, with scrollTop measurably stuck at 2000 through a direct assignment.
Fix	In this engine, smooth scrolling from either source drops the scroll entirely, not just the animation. Assigning scrollTop with no smooth behaviour anywhere is the only reliable form. The amber highlight does the orienting instead.
Found by	<i>Three wrong guesses, then measuring scrollTop directly</i>

THE PATTERN

Every one of these passed the test suite. Three needed the application to be run and looked at; the fourth needed a measurement rather than a hypothesis. On the citation bug specifically, three plausible fixes failed in a row and the measurement found the cause immediately — which is an argument for measuring earlier, not for guessing more carefully.

13

Status

What is verified, what is scaffolded, and what does not exist. Stated plainly because the distinction matters more than the total.

Verified — built, tested, and run

COMPONENT	HOW IT WAS VERIFIED
Domain model and rubric library	14 tests; every anchor checked for substance
Plan compiler and hashing	Hash stability and sensitivity both tested
Conductor	12 tests over transitions and both budgets
Evidence extraction and scoring	13 tests, plus end-to-end citation checking
Both report renderings	Rendered and read in terminal and browser
Text transport and CLI	Full interviews run end to end with no key
HTTP API	24 tests over the contract, including failure modes
Web video-call UI	Complete interviews driven through the browser in both modes
Persistence and history	12 tests; citation integrity verified after a SQLite round trip

Scaffolded — written, reviewed, never run

The realtime voice transport. It is written against livekit-agents 1.6.10 with the API surface verified by introspection rather than recalled, and the credential gate is tested. **The call path has never been executed against a live room**, because that needs LiveKit, speech-to-text, text-to-speech and avatar credentials. The module says so in its own docstring. Treat it as reviewed scaffolding.

Known limitations

- An interview **in progress** lives only in memory. Restarting the server loses it. Finished interviews are persisted; resuming a half-finished one is not supported.
- The keyless reasoner has real limits in both recall and precision. It can miss a strong answer whose anchors fall outside its six signals, and it can attach a genuine quote to the wrong critical point. Heuristic-mode levels are indicative, not defensible.
- There is no authentication, no multi-tenancy and no rate limiting. It is a personal tool bound to localhost.

Not built

The live-coding interview type. It needs a shared editor, a sandboxed runtime and code-aware scoring, and it sits awkwardly inside a talking-head video call. It was scoped out of the first cut deliberately rather than forgotten.

14

Running it

Everything below works with no API key.

Install

```
python3 -m venv .venv
.venv/bin/pip install -e ".[api,dev]"
cd web && npm install
```

Terminal

```
panel demo                                scripted, deterministic
panel practice --role "Backend Engineer" --minutes 20
panel screen  --role "Backend Engineer" --resume cv.txt --jd role.txt
panel history                                past runs and progress
```

Browser

```
./dev.sh           API on :8040, web app on :5193
```

Those ports were chosen to avoid the other projects already claiming 8000, 8001, 8010, 8020, 8030 and 5173. On another machine any free pair will do.

Optional credentials

VARIABLE	UNLOCKS
ANTHROPIC_API_KEY	The Claude reasoner. Without it the heuristic runs and everything still works
PANEL_MODEL	Model override; defaults to claude-opus-5
PANEL_DB	Database location; defaults to data/panel.db
LIVEKIT_URL / _API_KEY / _API_SECRET	Realtime voice
DEEPGRAM_API_KEY	Speech-to-text
CARTESIA_API_KEY	Text-to-speech
PANEL_AVATAR	tavus anam simli hedra none. Off by default

ON AVATAR COST

Talking-head avatars bill roughly \$0.10 to \$0.37 per active minute, so a 30-minute interview costs \$3 to \$11 in avatar alone. PANEL_AVATAR defaults to none for that reason, and missing credentials are reported by name with what each one buys.

15

Appendix

Commit history

HASH	WHAT LANDED
17e72fc	The engine: domain model, rubric library, plan compiler, conductor, extractor, scorer, both reports, text transport, CLI
94d0713	HTTP API and the video-call web UI; reports refactored into one object with several renderings
bcb4b9a	Realtime voice transport scaffolding behind a tested credential gate
f70646a	Append-only persistence, history, and trends scoped to one rubric

Across the last three commits: 31 files changed, 4,961 insertions.

Glossary

TERM	MEANING
Anchor	One level of a rubric, described as observable behaviour in an answer rather than as a trait
BARS	Behaviourally-anchored rating scale — the rubric form that lifts agreement with human experts from ≈ 0.20 to ≈ 0.63
Critical point	A concrete thing a strong answer contains; drives both probing and scoring
plan_hash	sha256 over the criteria, truncated to 16 characters. Two interviews sharing one were assessed identically
Probe	A follow-up question targeting the most valuable missing critical point
Coverage	Fraction of planned competencies that actually gathered evidence
Not observed	A competency the interview never reached. Excluded from the average; not a low score
Reasoner	The component making judgement calls. Two implementations: Claude-backed and keyless heuristic
Conductor	The interview state machine. Chooses which planned question and how deep to probe — never what to assess

Panel v0.1.0 · documentation generated 19 August 2026 · all figures measured from the repository at commit f70646a